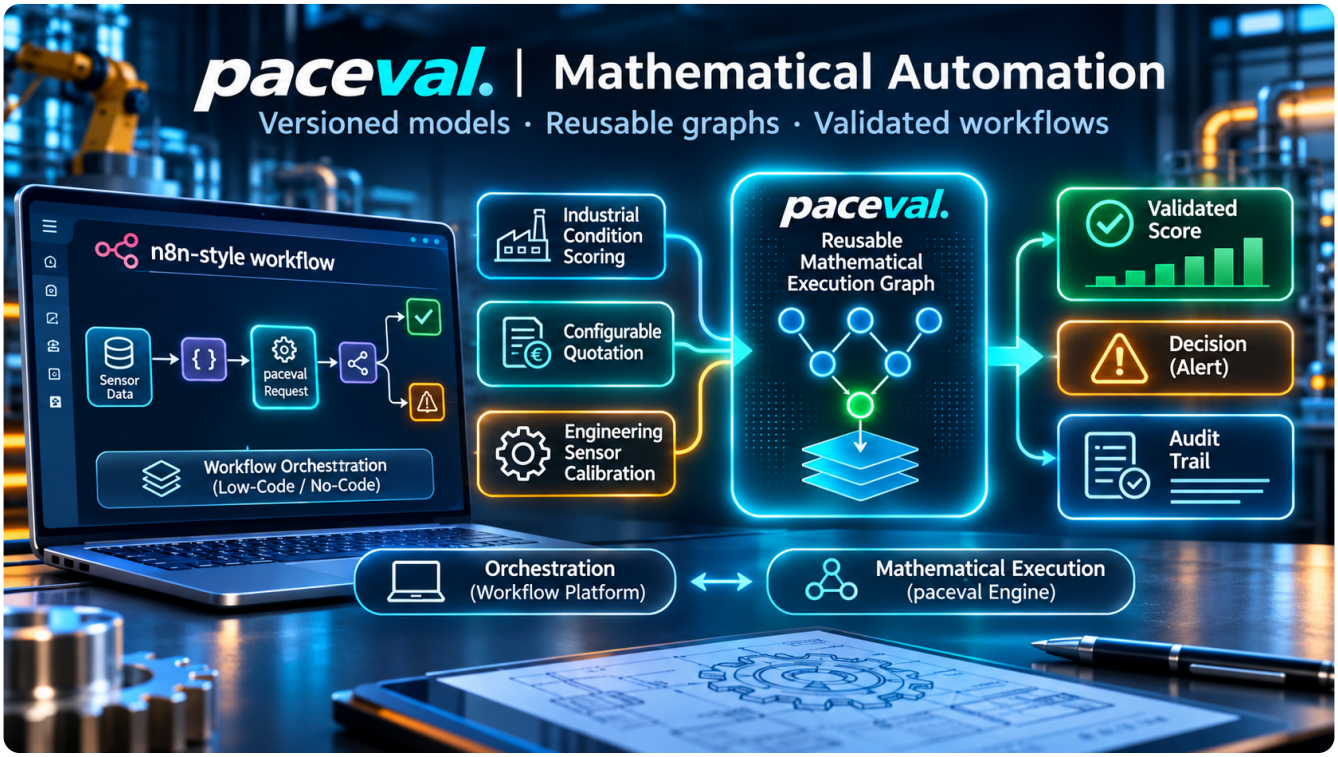
Reusable Mathematical Execution in Low-Code and No-Code Automation

A verified local paceval mathematical service for n8n-style workflow orchestration, approved model versions, batch calculations and independent numerical auditing.



Research implementation: C++17 on Linux using the original paceval 4.25 C/C++ API forwarding sources, linked to an externally supplied paceval runtime. This is a synthetic offline demonstrator, not a production deployment.

Executive research result

The attached ten-page research proposal framed the opportunity as a distinction between workflow orchestration and reusable mathematical model execution. This follow-up implements and validates that narrower boundary. Mathematical calculations are delegated to **paceval computation graphs**; input contracts, HTTP serialization, scenario selection and workflow decisions remain outside the engine.

4 + 1	**320 / 320**	**24 / 24**
Approved model graphs plus one pointwise TINC graph	Published numerical results within the test tolerance	Selected TINC intervals contain the 80-digit oracle

The original project question is answered positively **for the specified mathematical execution layer**. The local persistent HTTP bridge was tested using JSON requests, a reused C++ process and the real paceval runtime. Five n8n workflow definitions were generated and checked structurally, but were **not executed within a live n8n installation**. No general speedup or production n8n compatibility is claimed.

1. Existing software and the exact boundary

n8n already includes arithmetic expressions, JavaScript/Python Code nodes, an HTTP Request node and an AI-agent Calculator. Therefore, the compelling pilot is not multiplication or percentages. It is the governance of explicit

mathematical models: one formula with controlled variable order, a documented version, consistent domain rules and reuse in several otherwise independent workflows. The host platform still owns business logic and authorization. [1-3]

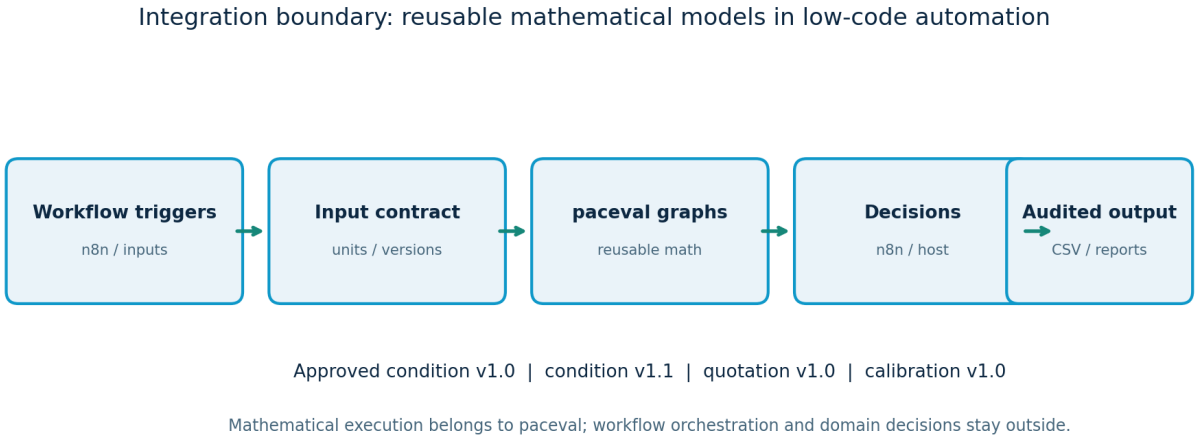


Figure 1. Tested local integration boundary. The n8n boxes describe the intended orchestration; the runtime was exercised through the local Python HTTP adapter, not a live n8n installation.

Why an intermediary? The official C/C++ paceval API exposes computation handles and batch evaluation; for an n8n workflow, an HTTP adapter provides a normal JSON request/response contract. The example bridge binds only to loopback, invokes a persistent C++ process, and exposes fixed approved models instead of accepting untrusted formula uploads. This adapter is a **new local research implementation**, not a representation of the paceval cloud REST service.

Other integration routes

n8n has documented MCP Client / MCP Client Tool capabilities, and paceval publishes a Streamable HTTP MCP endpoint. Their current transport, handshake, credentials and selected node-version interoperability require separate live verification. The included project does not claim a successful n8n-to-paceval MCP call or an AI agent using MCP. [4-6]

2. Implemented mathematical models

The following formulas are the explicit subproblems proposed in the source paper, plus a controlled version update to demonstrate governance. Values and coefficients are synthetic and normalized as documented in the source code.

Approved ID / version	Expression	Contract
condition_score 1.0	`S = 1/(1+exp(-(b0+bT*uT+bV*uV+bI*uI)))`	7 finite inputs; uT,uV,uI are normalized to [-1,1]
condition_score 1.1	v1.0 with `+0.05*uT*uV` in the linear predictor	A separate immutable graph, not an in-place overwrite
quotation 1.0	`Q=C*(1+m)*(1+r)^t+F`	5 inputs; valid positive factors and bounded term
calibration 1.0	`Tcorr=T+a0+a1*T+a2*T*T+b*ln(1+h)`	6 inputs; h is nonnegative and range checked

Scope of the models. The logistic score is not a trained failure probability. The quote does not encode tax, contract, exchange-rate or pricing approval requirements. The calibration model is not fit to a physical sensor. Each is a transparent mathematical kernel for validating the integration and numerical computation processes.

3. Numerical studies and model versions

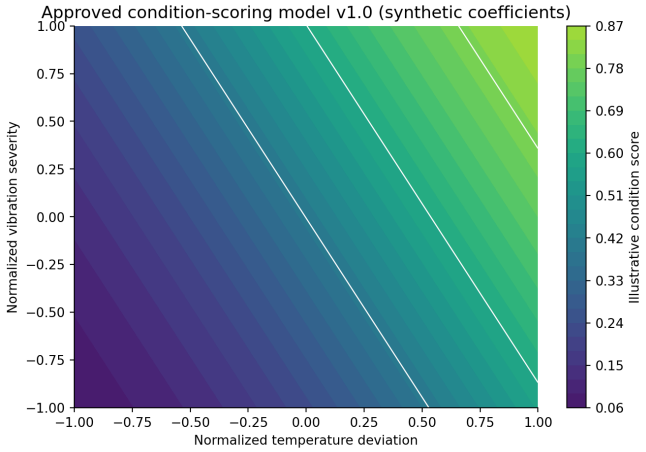


Figure 2. Response surface of a synthetic condition model over normalized sensor deviations.

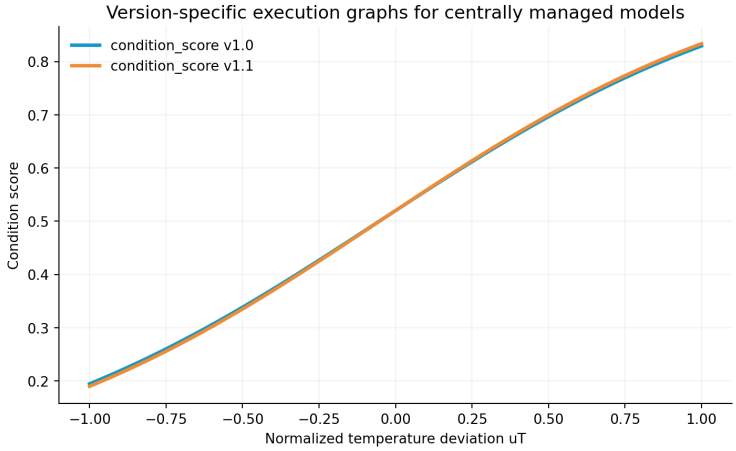


Figure 3. Same model identity, distinct approved versions with independently created paceval handles.

The model identifier is a curated string, not a user-supplied expression. A caller cannot silently change the meaning of model 1.0 by providing alternative mathematical text. Publication of a new version requires a code/catalog revision and revalidation. In the example files, two separate condition workflows use the exact same model/version.

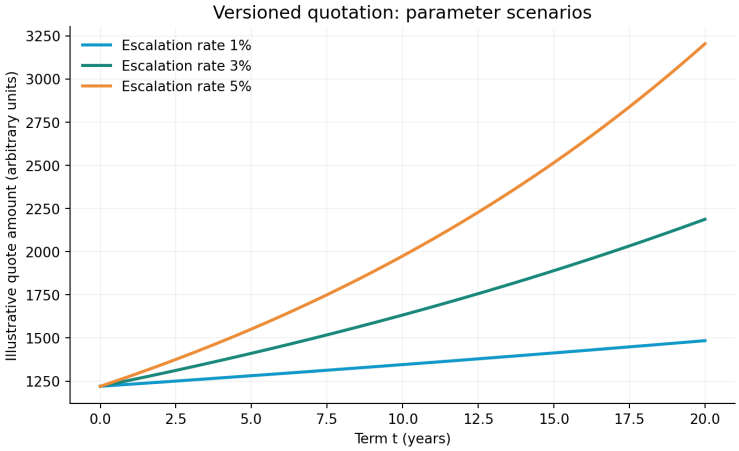


Figure 4. Parameterized quotations using the approved expression. Values are illustrative.

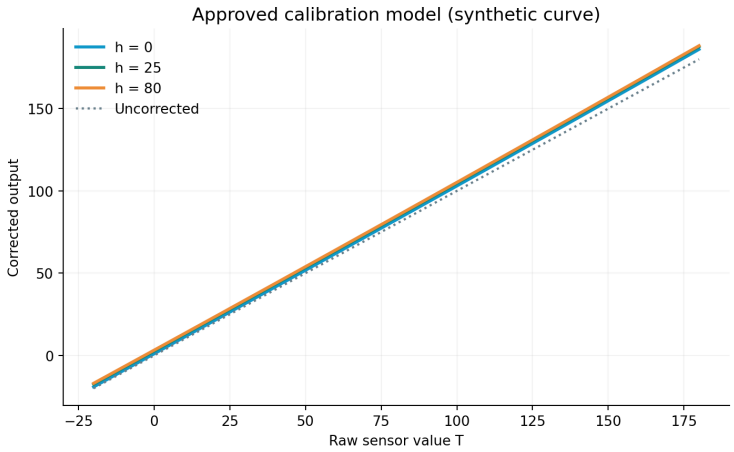


Figure 5. Synthetic correction model under differing humidity-related inputs.

4. What was actually implemented with paceval

SDK integration. The uploaded official `paceval_main.h` and `paceval_main.cpp` API layer is compiled with `src/lowcode_math.cpp` and linked to the separately supplied Linux runtime. The mathematical engine itself is not replaced by the forwarding source file. No custom `dlopen`/`dlsym` function table is used.

SDK function	Actual role
`paceval_InitializeLibrary`, `paceval_FreeLibrary`	Engine initialization and orderly shutdown
`paceval_CreateComputation`	Four approved reusable graphs plus one interval-enabled graph
`paceval_dGetComputationResultExt`	Batch evaluation of sample-major arrays under one graph
`paceval_dGetComputationResult`	Pointwise TINC sample evaluation
`paceval_GetIsError` and per-result error codes	Detect graph errors and rejected numerical calculations
`paceval_DeleteComputation`	Release every computation before freeing the library

The process remains alive in `--stdio` mode and owns the prepared model objects. The Python `http_bridge.py` process serializes approved JSON requests into a strict local line protocol and returns structured results. For a query model/version that has already been prepared, the same graph is reused across subsequent client requests.

The host rejects nonfinite numbers, wrong counts, unknown model versions and out-of-contract values before issuing the calculation. The application limit of 512 rows per HTTP request is a configurable defensive policy of this demonstrator, **not a hard paceval expression/variable/batch limit**.

5. Independent scientific verification

The C++ source uses a separately written native analytic function per model as its immediate comparison oracle. For each of four approved graph versions, 80 deterministic value sets were evaluated and checked: 320 mathematical outputs in total. All met the specified absolute/relative tolerance `3e-11 * max(1, abs(reference))` in the independent 80-digit audit.

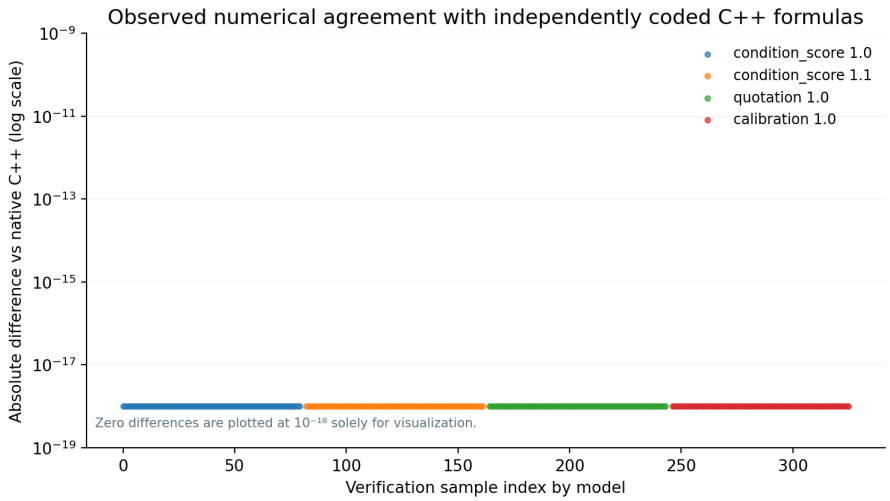


Figure 6. Native C++ comparisons across all 320 samples; exact zeros are visually floored only for logarithmic plotting.

A second implementation, `src/high_precision_reference.py`, computes the expressions with 80-digit Python Decimal arithmetic, including exponential/logarithmic terms. The maximum absolute difference in this sample is approximately **3.8 × 10^-11** (on the larger quotation outputs). The verification is performed on the output CSV, not on internally reused paceval results.

For 24 condition-score inputs, an additional interval-enabled graph supplied TINC lower and upper values. All 24 independently calculated 80-digit references lay within their respective reported intervals in this selected sample. This is **sample-specific numerical evidence**, not a universal mathematical guarantee for arbitrary functions, compilers or environments.

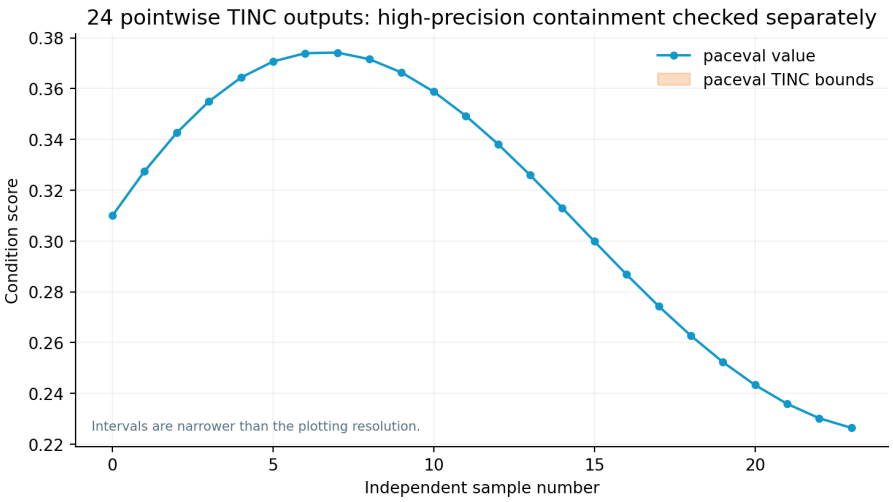


Figure 7. Selected pointwise interval outputs; numeric widths are smaller than the display resolution.

6. Workflow and service experiments

The local research service provides three routes: `GET /health`, `GET /api/v1/models` and `POST /api/v1/evaluate`. The POST body uses the exact model ID, version and an array of value arrays in the documented order.

```
{"model":"condition_score","version":"1.0","rows":[[0,0,0,0,1,1,1],[0.5,0.1,-0.2,0,1,1,1]]}
```

The HTTP integration test runs a real child process with the original paceval runtime and checks the correct result, repeated-request equality, version isolation, quotation, calibration and rejection of wrong versions, NaN, invalid ranges or incorrect row widths. Five sample n8n workflow files were constructed and their JSON syntax and graph connections validated independently. These are **templates**: runtime import compatibility still requires a real n8n installation.

The fifth workflow evaluates the identical condition formula in JavaScript through n8n's Code node, serving as a fair native mathematical reference. The research result is version-controlled execution and a reusable service boundary, not superiority over n8n's existing calculator.

Security and deployment boundary

- The bridge only binds to `127.0.0.1` by default; it has no public authentication/TLS solution.
- Model expressions are compiled into a fixed allowlist. The HTTP request cannot upload arbitrary untrusted formulas.
- Future production work requires tenant authorization, worker restarts, rate limits, durable model registry, error/retry policy, observability, and controlled credential storage.
- n8n's October 2026 licensing guidance distinguishes workflows authored within one organization from customer-editable/hosted workflow products; embedding should be reviewed separately. [7]

7. Reproducibility and limitations

The tested environment provided Linux x86-64, GCC 14.2, C++17, CMake and the original external paceval Linux runtime. The project is also tested with Clang and sanitizers where the reproducer is available. The documentation contains no speed, energy or FPGA acceleration measurements; these would need a separately controlled end-to-end study.

Included artifact	Role
`README.md`	Instructions, figure links, source references and scope
`src/lowcode_math.cpp`, `src/CMakeLists.txt`	Direct SDK integration, native audit and persistent model process
`src/paceval_main.h`, `src/paceval_main.cpp`	Original paceval API forwarding source and embedded AGPL notices
`src/http_bridge.py`, `src/test_http_bridge.py`	Loopback JSON transport and real process test
`src/high_precision_reference.py`	Independent 80-digit model and TINC calculation
`examples/batch_results.csv`, `examples/tinc_samples.csv`	Frozen, independently reproducible validation records
`workflows/01_condition_scoring_alert.json`, `workflows/02_condition_dashboard_same_version.json`, `workflows/03_quotation.json`, `workflows/04_calibration.json`, `workflows/05_native_js_condition_reference.json`	Import-ready n8n examples (not live-tested)

Limitations are deliberately explicit: no live n8n instance or MCP tool handshake was tested; there is no physically trained industrial failure model, no legally binding price calculation, and no real sensor calibration. The project demonstrates feasibility of the mathematical execution layer and a compatible local HTTP request/response design under synthetic test data.

8. Conclusions and next research steps

The results show that approved mathematical model versions can be compiled once and reused through the original paceval SDK while low-code processes retain orchestration and domain decisions. The numerical checks support correct evaluation of the tested models, and the local HTTP bridge makes a practical deterministic integration boundary available for further work.

Next steps are to import and execute the sample workflows in a pinned n8n release, compare domain decisions and maintenance across the real native-expression, Code and paceval paths, test service recovery/credentials/network failure modes, and independently evaluate the MCP agent route. Hardware acceleration is a separate optional architecture hypothesis rather than an experimental result.

> **Research outcome:** A complete, independently checked mathematical execution pilot has been implemented with paceval. Actual n8n end-to-end interoperability and any product advantages must be established in a future live integration study.

References and implementation sources

[1] n8n HTTP Request node. docs.n8n.io/integrations/builtin/core-nodes/n8n-nodes-base.httprequest/

[2] n8n Code node. docs.n8n.io/integrations/builtin/core-nodes/n8n-nodes-base.code/

[3] n8n Calculator tool. docs.n8n.io/integrations/builtin/cluster-nodes/sub-nodes/n8n-nodes-langchain.toolcalculator/

[4] n8n MCP Client Tool. docs.n8n.io/integrations/builtin/cluster-nodes/sub-nodes/n8n-nodes-langchain.toolmcp/

[5] paceval C/C++ API 4.25. paceval.com/api/

[6] paceval MCP server and public repository. github.com/paceval/paceval

[7] n8n licensing FAQ, updated 6 October 2026. support.n8n.io/article/can-i-use-your-license-for-my-use-case

Primary starting point: *Reusable Mathematical Execution in Low-Code and No-Code Automation*, uploaded research proposal, 11 November 2026. The original proposal is not included in this minimal release; this report replaces it with implemented experimental results.